
COMPLEX ENGINEERING PROBLEM

TITLE

Git Branching and Merge-Conflict Resolution

Maitri Adroja (92400103446)

Krupa Makwana (92400103408)

Manthan Makwana (92400103407)

DEPARTMENT

COMPUTER ENGINEERING

ACADEMIC YEAR

2026-2027

SEMESTER

5TH

SUBJECT

OPEN SOURCE TECHNOLOGIES(01CE0526)

CLASS

5EC3

BATCH

A

COMPLEX ENGINEERING PROBLEM - REPORT

RUBRICS FOR COMPLEX ENGINEERING PROBLEM

Sr. No.	Assessment Criteria	Excellent	Good	Satisfactory	Poor	Marks
1	Problem Analysis & Open Source Technology Selection	3 Marks: Clearly identifies the engineering problem, analyzes requirements and constraints, and selects appropriate Open Source technologies with proper justification.	2 Marks: Good problem analysis with suitable technology selection and minor gaps.	1 Mark: Basic analysis with limited justification of technology selection.	0 Marks: Inadequate problem analysis and inappropriate technology selection.	3
2	System Design, Implementation Strategy & Technical Analysis	4 Marks: Well-defined system architecture, appropriate technology stack, database, deployment strategy, version control, and comprehensive technical analysis including scalability, reliability, and security.	3 Marks: Good system design and technical discussion with minor omissions.	1-2 Marks: Basic architecture and limited technical analysis.	0 Marks: Poor or missing system design and technical analysis.	4
3	Testing, Documentation & References	3 Marks: Comprehensive testing strategy, well-organized report, IEEE references, appropriate diagrams, and clear conclusions.	2 Marks: Good documentation with minor formatting or testing gaps.	1 Mark: Basic report with limited testing or references.	0 Marks: Incomplete report with poor documentation and missing references.	3
	Total					10

Sr. No.	Complex Engineering Problem Activity / Problem Title	COs	Date	R1(4)	R2(3)	R3(4)	Marks (10)	Signature
1	Git Branching and Merge-Conflict Resolution							

Notes / Instructions

1. CEP activity must be completed in a group of maximum 3 students.
2. Students must strictly follow the given deadlines for submission.
3. The report must be prepared in the prescribed format and submitted as a DOCX file on Google Classroom.
4. A hard copy of the report must be submitted as discussed.
5. The report should include problem analysis, design/architecture, implementation, results, and challenges faced.
6. All group members must contribute equally and be prepared for evaluation/presentation.
7. Plagiarism or copying from other groups will not be accepted.
8. Each group member must understand the complete solution and should be able to explain their contribution during evaluation.

Ms. Priyanka Mangi
Subject Faculty

COMPLEX ENGINEERING PROBLEM - REPORT

Course Name and Course Code: OPEN SOURCE TECHNOLOGIES(01CE0526)

Sr. No.	Problem Title	COs Mapped	SDGs Mapped
1	Git Branching and Merge-Conflict Resolution	CO2	SDG 4, SDG 9, SDG 17

1. Aim

In order to analyze, design and implement a version control system based on Git that will generate conflicts for a shared file between two branches and solve the created conflict in a proper way.

2. Problem Statement

Create a repository with main and feature branches. Make conflicting changes in the same file, merge the branches, resolve the conflict, and document the resolution steps.

3. Problem Analysis

a) What is the engineering problem?

In collaborative software development, multiple developers may work on the same project simultaneously. When different developers modify the same portion of a file on separate Git branches, Git may be unable to automatically combine the changes, resulting in a merge conflict.

The engineering problem is to correctly identify the conflicting changes, analyse the required final version, resolve the conflict without losing necessary work, and verify that the repository is in a consistent state.

b) Why is this problem complex?

The problem involves the following engineering aspects:

- ☒ Requires multiple engineering concepts
- ☒ Requires analysis and decision making
- ☒ Involves design constraints
- ☒ Requires use of engineering tools
- ☒ Requires reliability and consistency consideration

The complexity mainly occurs because the developer must understand branch history, identify conflicting modifications, decide which content should be retained, manually resolve the conflict, and verify the resulting repository.

COMPLEX ENGINEERING PROBLEM - REPORT

Real-World Application Example

Consider an online shopping company that is developing an e-commerce application. One developer is working on a new payment-related feature inside a feature branch, while another developer is maintaining the stable production code in the main branch. Suppose both developers modify the same portion of a configuration file that stores the application's operating mode.

Original file content:

Application Mode: Basic

Change made on the feature branch:

Application Mode: Development

Change made on the main branch:

Application Mode: Production

When the feature branch is merged into main, Git detects that the same line of the same file was changed differently in both branches and generates a merge conflict. The developer must inspect both versions, understand the purpose of each change, and decide the correct final content. After resolving the conflict, the developer stages the resolved file and creates a merge commit. The final repository is then verified using git status and git log, and the changes are pushed to GitHub.

Real-World Significance:

The Git branching and merge-conflict resolution workflow demonstrated in this report is a general software-engineering practice and is not limited to any single domain. The same principle applies wherever multiple developers work independently and their changes must later be integrated – for example, in e-commerce applications, banking and payment systems, hospital management systems, college management systems, general web application development, and team-based software projects of any size. In each of these cases, version control provides a controlled mechanism through which independent contributions can be integrated and conflicts can be resolved without losing any developer's work.

4. Knowledge and Tools Required

Engineering concepts used:

1. Version control systems and distributed repository models.
2. Branching and merging strategies in software development.
3. File-level diffing and conflict-marker interpretation.

Tools/Software/Hardware used:

1. Git (open-source distributed version control system).
2. Git Bash / terminal for command execution.
3. A text/code editor (e.g., VS Code) for editing files and resolving conflicts.
4. GitHub (optional) for hosting the remote repository.

Open-Source Technology Selection and Justification

Git is used as the primary open-source technology for this problem. Git is a free and open-source distributed version control system released under the GNU General Public License, which allows every

COMPLEX ENGINEERING PROBLEM - REPORT

developer to maintain a full local copy of the project history and work offline. It is important to note that Git itself is open-source, whereas GitHub is a proprietary hosting platform built around Git; GitHub is used here only as an optional remote hosting service and is not itself classified as open-source. Git is appropriate for this problem because it provides native branching, three-way merge algorithms, and clear conflict markers, which are exactly the capabilities required to demonstrate branch isolation, conflicting edits, and manual conflict resolution.

5. Proposed Solution Design

Block Diagram / Architecture Diagram:

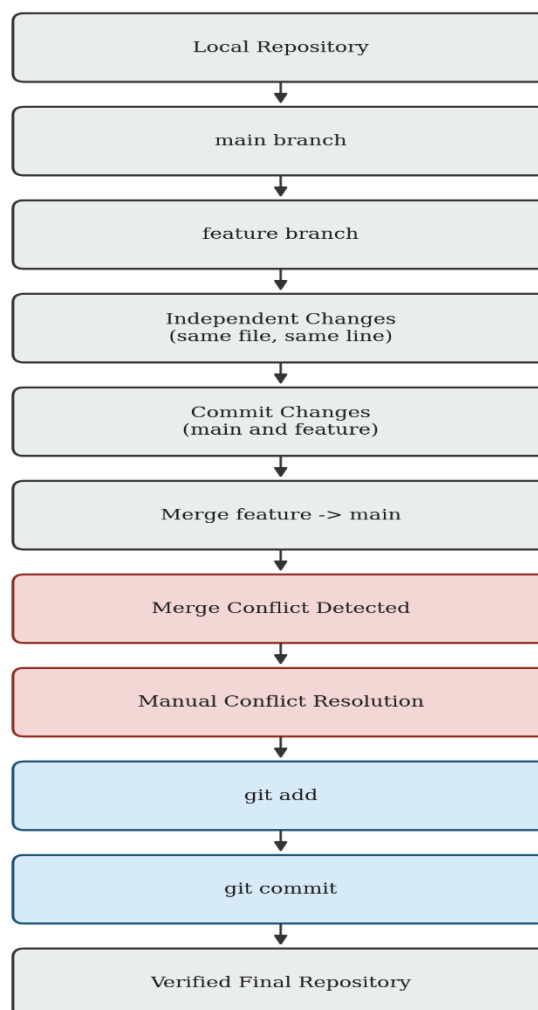


Figure 1: Git Branching and Merge-Conflict Resolution Workflow

Brief Working:

This process starts with the local Git repository where there is a main branch present. The first step of this process is creating a feature branch from the main branch. In both branches, changes are applied to the same line in the same tracked file. At the time when the feature branch is merged into the main

COMPLEX ENGINEERING PROBLEM - REPORT

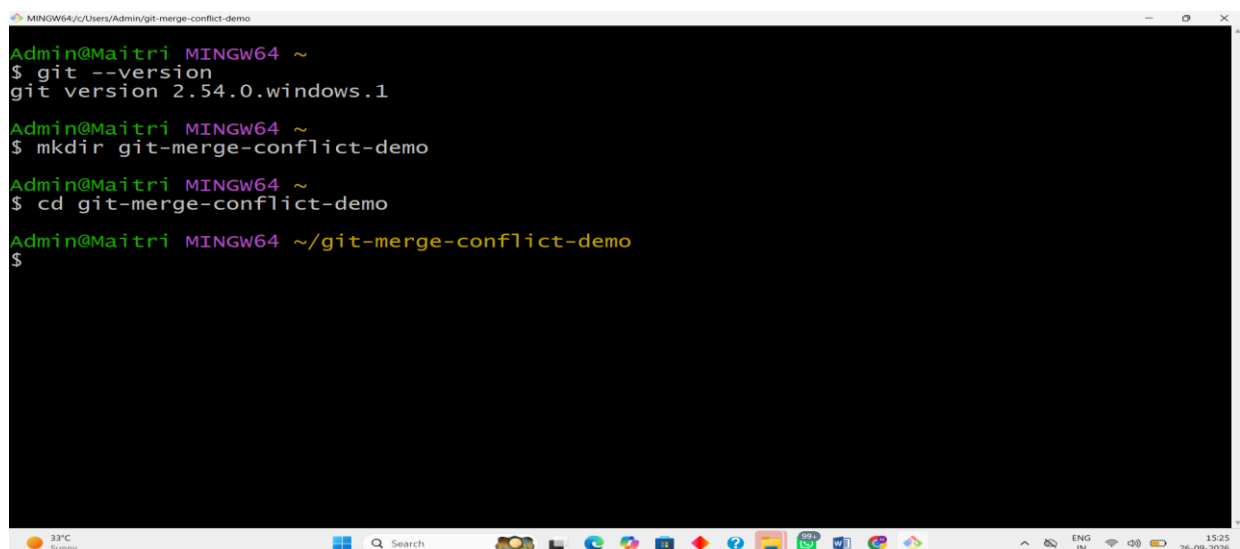
branch, Git is able to determine that the same line was changed in both branches and it could not be automatically merged. The process of merging stops at that point and Git puts conflict markers in that area of the file. Conflicts are inspected by the developer and the decision about the final content of that area is made by the developer. After that, the conflict markers are deleted and git add is used for staging the resolved file and git commit is used for completing the merge process.

6. Implementation / Procedure

Step 1: Check Git installation and create a working folder

```
$ git --version
$ mkdir git-merge-conflict-demo
$ cd git-merge-conflict-demo
```

The first command confirms that Git is installed and displays its version. The mkdir command creates a new empty folder for the demonstration, and cd moves into that folder so that all further commands operate inside it.



```
Admin@Maitri MINGW64 ~
$ git --version
git version 2.54.0.windows.1

Admin@Maitri MINGW64 ~
$ mkdir git-merge-conflict-demo

Admin@Maitri MINGW64 ~
$ cd git-merge-conflict-demo

Admin@Maitri MINGW64 ~/git-merge-conflict-demo
$
```

Figure 2: Git Installation Verification and Working Directory Creation

Step 2: Initialize the repository and set the main branch

```
$ git init
$ git branch -M main
```

git init initializes a new, empty Git repository inside the working folder by creating a hidden .git directory that stores all version-control data. git branch -M main renames the default branch to main, which is the current standard convention for the primary branch.

COMPLEX ENGINEERING PROBLEM - REPORT

```
Admin@Maitri MINGW64 ~/git-merge-conflict-demo
$ git init
Initialized empty Git repository in C:/Users/Admin/git-merge-conflict-demo/.git/

Admin@Maitri MINGW64 ~/git-merge-conflict-demo (master)
$ git branch -M main

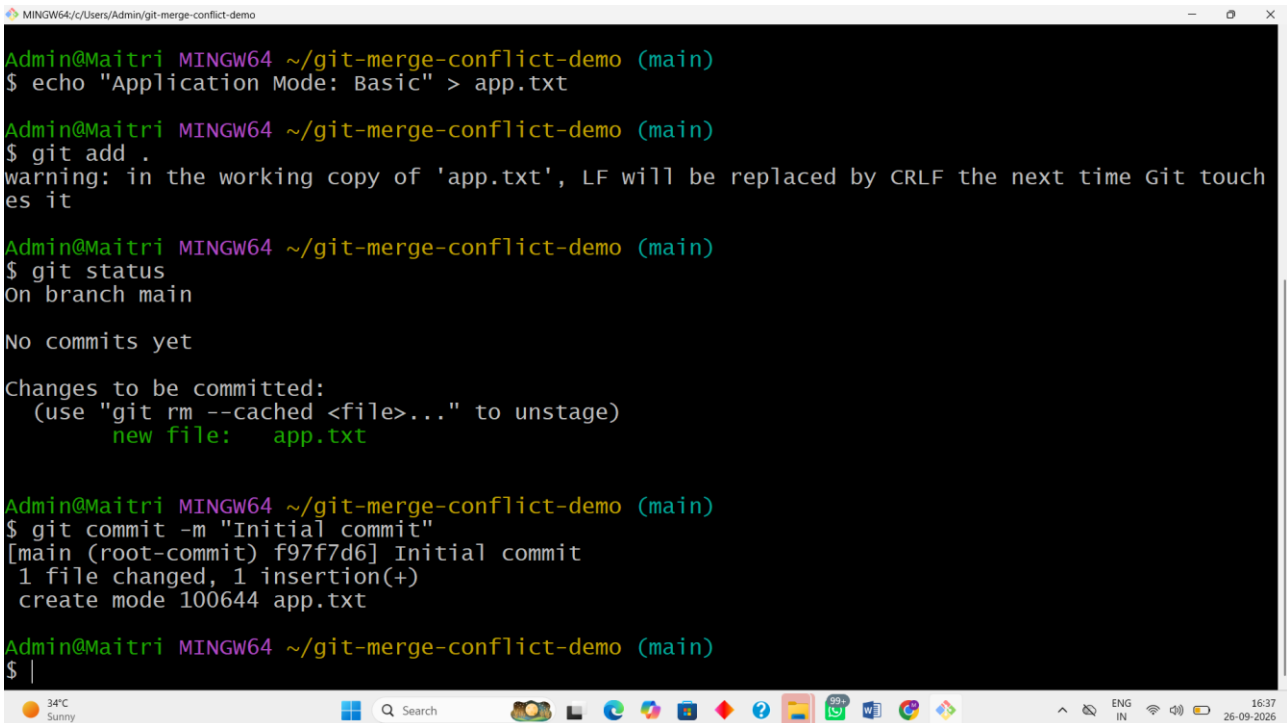
Admin@Maitri MINGW64 ~/git-merge-conflict-demo (main)
$ |
```

Figure 3: Git Repository Initialization

Step 3: Create app.txt and make the initial commit

```
echo "Application Mode: Basic" > app.txt
git add .
git status
git commit -m "Initial commit"
```

The echo command creates app.txt with the starting content. git add . stages the new file, git status confirms that it is staged, and git commit records it as the first commit on the main branch. This commit becomes the common ancestor from which both the main and feature branches will later diverge.



```
Admin@Maitri MINGW64 ~/git-merge-conflict-demo (main)
$ echo "Application Mode: Basic" > app.txt

Admin@Maitri MINGW64 ~/git-merge-conflict-demo (main)
$ git add .
warning: in the working copy of 'app.txt', LF will be replaced by CRLF the next time Git touches it

Admin@Maitri MINGW64 ~/git-merge-conflict-demo (main)
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   app.txt

Admin@Maitri MINGW64 ~/git-merge-conflict-demo (main)
$ git commit -m "Initial commit"
[main (root-commit) f97f7d6] Initial commit
1 file changed, 1 insertion(+)
create mode 100644 app.txt

Admin@Maitri MINGW64 ~/git-merge-conflict-demo (main)
$ |
```

Figure 4: Initial Commit on Main Branch

COMPLEX ENGINEERING PROBLEM - REPORT

Step 4: Create the feature branch

```
git checkout -b feature
```

This command creates a new branch named feature starting from the current state of main, and switches to it immediately. All work done on feature from this point onward does not affect main until the branches are merged.

```
Admin@Maitri MINGW64 ~/git-merge-conflict-demo (main)
$ git checkout -b feature
Switched to a new branch 'feature'

Admin@Maitri MINGW64 ~/git-merge-conflict-demo (feature)
$ |
```

Figure 5: Feature Branch Creation

Step 5: Make conflicting changes on feature and main

On the feature branch, app.txt was edited to change the same line:

```
Application Mode: Development
```

```
git add app.txt
git commit -m "Update mode to Development on feature branch"
```

The branch was then switched back to main:

```
git checkout main
```

On the main branch, the same line in app.txt was changed to a different value:

```
Application Mode: Production
```

```
git add app.txt
git commit -m "Update mode to Production on main branch"
```

Both branches have now modified the same line of app.txt in two different ways, starting from the same common ancestor commit. This is exactly the condition under which Git raises a genuine content conflict during a merge, since it has no way of deciding automatically which line should be kept.

COMPLEX ENGINEERING PROBLEM - REPORT

```
Admin@Maitri MINGW64 ~/git-merge-conflict-demo (feature)
$ git add app.txt

Admin@Maitri MINGW64 ~/git-merge-conflict-demo (feature)
$ git commit -m "Update mode to Development on feature branch"
On branch feature
nothing to commit, working tree clean

Admin@Maitri MINGW64 ~/git-merge-conflict-demo (feature)
$ git checkout main
Switched to branch 'main'

Admin@Maitri MINGW64 ~/git-merge-conflict-demo (main)
$ git add app.txt

Admin@Maitri MINGW64 ~/git-merge-conflict-demo (main)
$ git commit -m "Update mode to Production on main branch"
On branch main
nothing to commit, working tree clean

Admin@Maitri MINGW64 ~/git-merge-conflict-demo (main)
$ |
```

Figure 6: Conflicting Changes on Main and Feature Branches

Step 6: Merge feature into main

```
git checkout main
git merge feature
```

Git attempts an automatic merge, but since the same line was modified differently in both branches, it cannot choose one version over the other and reports a conflict:

```
CONFLICT (content): Merge conflict in app.txt
Automatic merge failed; fix conflicts and then commit the result.
```

```
Admin@Maitri MINGW64 ~/git-merge-conflict-demo (feature)
$ git checkout main
Switched to branch 'main'

Admin@Maitri MINGW64 ~/git-merge-conflict-demo (main)
$ git add app.txt

Admin@Maitri MINGW64 ~/git-merge-conflict-demo (main)
$ git commit -m "Update mode to Production on main branch"
[main f2578a9] Update mode to Production on main branch
1 file changed, 1 insertion(+)

Admin@Maitri MINGW64 ~/git-merge-conflict-demo (main)
$ git merge feature
Auto-merging app.txt
CONFLICT (content): Merge conflict in app.txt
Automatic merge failed; fix conflicts and then commit the result.

Admin@Maitri MINGW64 ~/git-merge-conflict-demo (main|MERGING)
$
```

Figure 7: Merge Conflict Generated by Git

COMPLEX ENGINEERING PROBLEM - REPORT

Step 7: Inspect conflict markers

Opening app.txt shows the conflict-marker structure that Git inserts into the file:

```
<<<<<< HEAD
Application Mode: Production
=====
Application Mode: Development
>>>>>> feature
```

<<<<<< HEAD marks the beginning of the current branch's version (main). The ===== line separates the two conflicting versions. >>>>>> feature marks the end of the block and identifies the incoming branch whose version is shown below the separator.

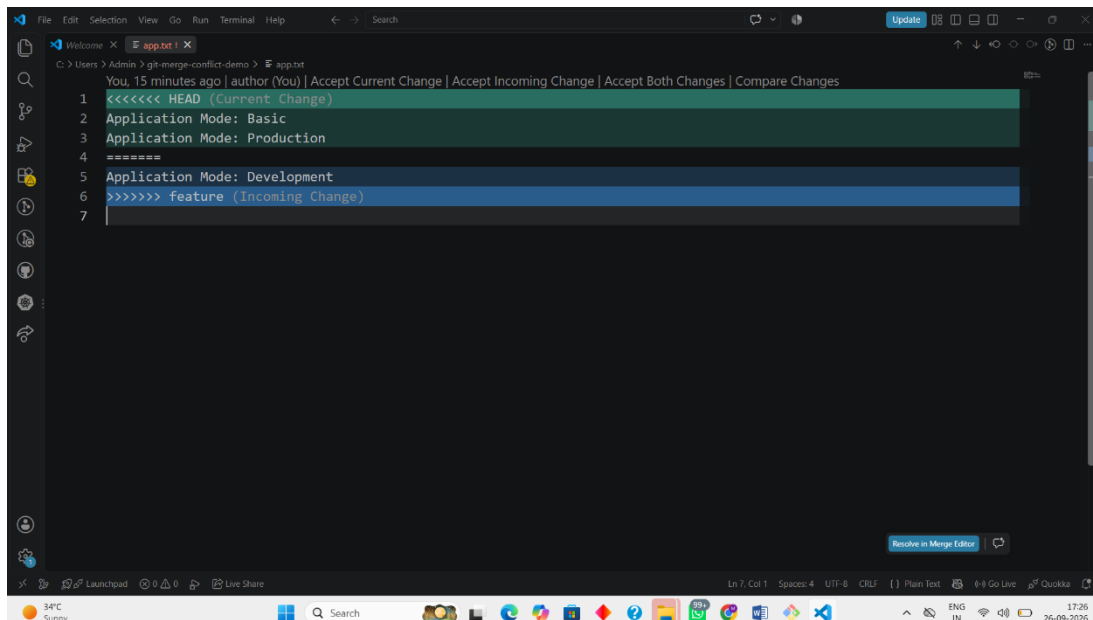


Figure 8: Conflict Markers in the Source File

Step 8: Resolve the conflict

Both versions were reviewed before deciding on the final content. Since the main branch represents the stable production version of the project, the Production value was retained as the final resolved content, and the conflict markers, along with the discarded Development line, were removed. The resolved file was saved with the following content:

```
Application Mode: Production
```

This decision reflects a deliberate engineering choice: in a real project, the production configuration on the main branch should not be silently overwritten by a value intended only for a development branch, so the Production setting was kept as the final result.

COMPLEX ENGINEERING PROBLEM - REPORT

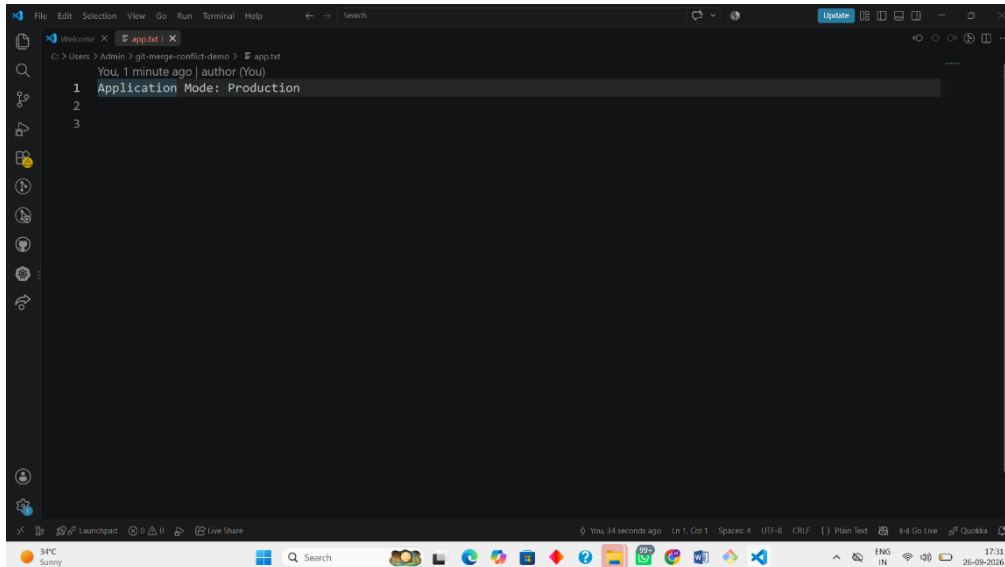


Figure 9: Conflict Resolution in the Source File

Step 9: Stage and complete the merge

```
git add app.txt
git commit -m "Resolve merge conflict between main and feature"
```

git add app.txt stages the resolved file, marking the conflict as handled. git commit then completes the merge by creating a merge commit that has both the main and feature branch commits as its parents.

```
Admin@Maitri MINGW64 ~/git-merge-conflict-demo (main|MERGING)
$ git add app.txt

Admin@Maitri MINGW64 ~/git-merge-conflict-demo (main|MERGING)
$ git commit -m "Resolve merge conflict between main and feature"
[main e559598] Resolve merge conflict between main and feature

Admin@Maitri MINGW64 ~/git-merge-conflict-demo (main)
$ |
```

Figure 10: Successful Merge Commit

Step 10: Verify repository status and commit history

```
git status
git log --oneline --graph --all
```

git status confirms that no unresolved conflict remains:

```
On branch main
nothing to commit, working tree clean
```

COMPLEX ENGINEERING PROBLEM - REPORT

git log --oneline --graph --all displays the full commit history, including the point where the feature branch diverged from main and the commit where the two branches were merged back together, giving complete traceability of the changes.

```
Admin@Maitri MINGW64 ~/git-merge-conflict-demo (main)
$ git status
On branch main
nothing to commit, working tree clean

Admin@Maitri MINGW64 ~/git-merge-conflict-demo (main)
$ git log --oneline --graph --all
* e559598 (HEAD -> main) Resolve merge conflict between main and feature
| \
| * 9ecc9ea (feature) Update mode to Development on feature branch
| | f2578a9 Update mode to Production on main branch
| |
|/
* f97f7d6 Initial commit

Admin@Maitri MINGW64 ~/git-merge-conflict-demo (main)
$
```

Figure 11: Git Commit History and Repository Status

Step 11: Push the final repository to GitHub

```
git remote add origin <repository-url>
git push -u origin main
```

git remote add origin links the local repository to a remote repository hosted on GitHub, and git push -u origin main uploads the local main branch, along with its complete commit history, to that remote repository so that it can be viewed and shared online.

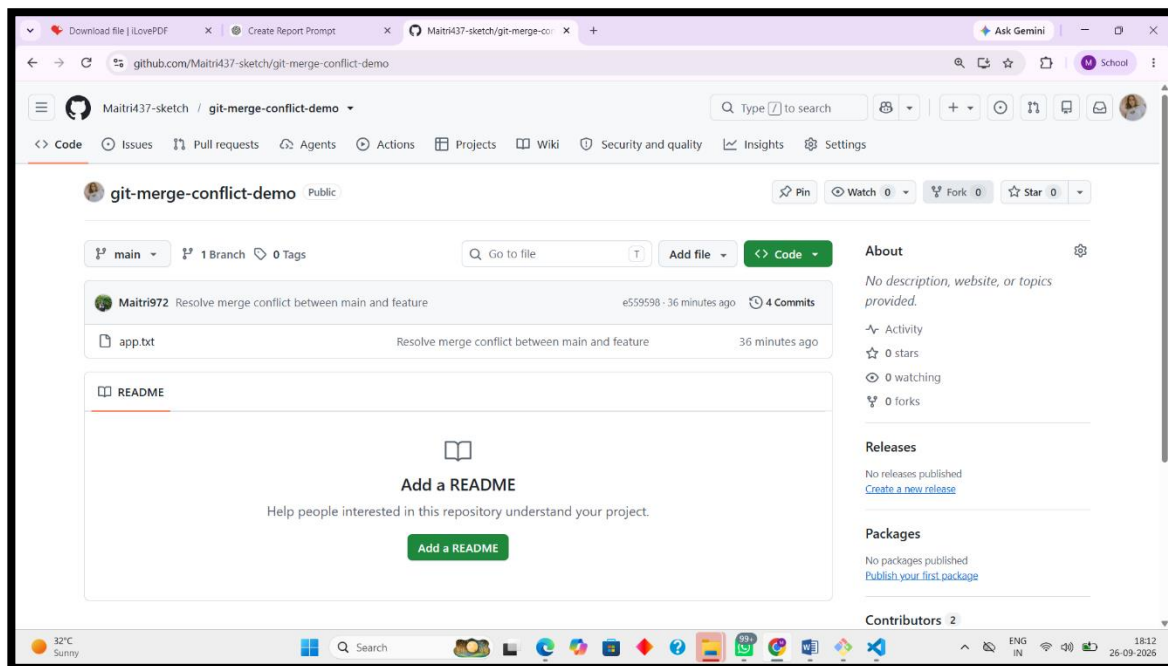


Figure 12: Final GitHub Repository

COMPLEX ENGINEERING PROBLEM - REPORT

GitHub Repository and Commit History

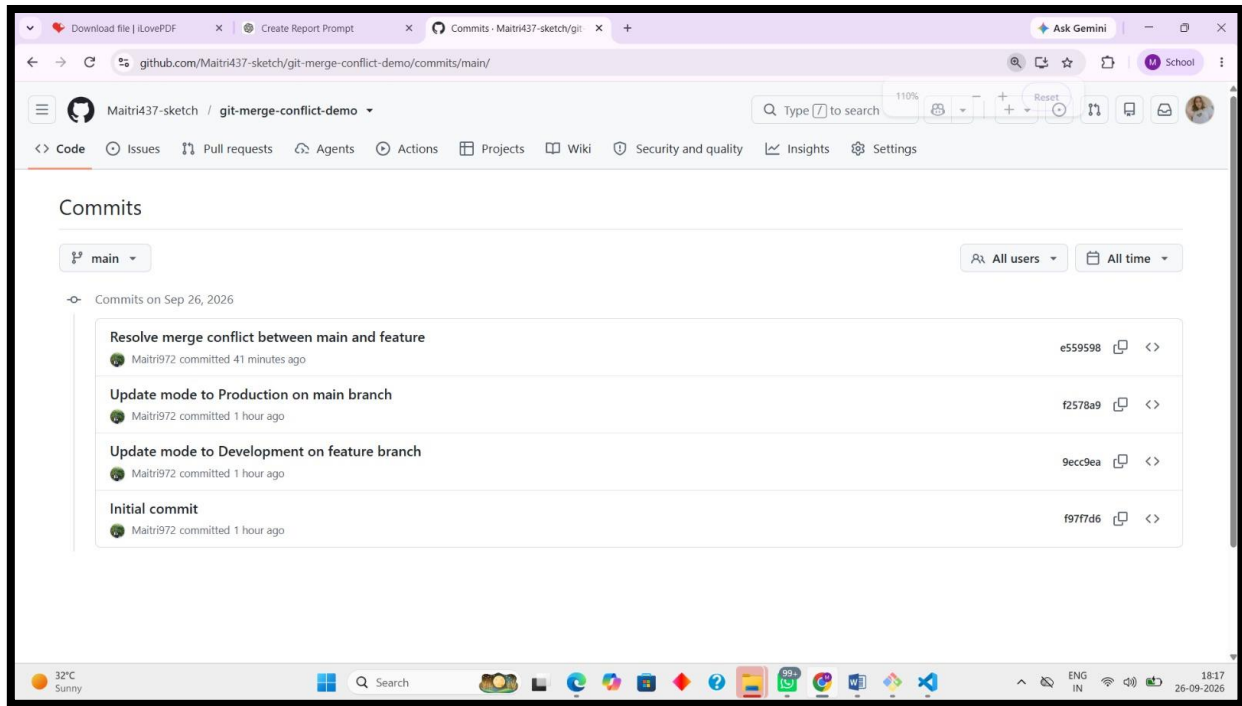


Figure 13: GitHub Commit History

The commit history on GitHub should show, in order: the initial commit, the Development change made on the feature branch, the Production change made on the main branch, and the final commit that resolved the merge conflict.

7. Testing and Results

Sr. No.	Test / Parameter	Expected Result	Obtained Result	Status
1	Git installation verification	Git version displayed	Git version displayed successfully	Pass
2	Repository initialization	Empty repository created	.git folder created in working directory	Pass
3	Main branch creation	Branch renamed to main	Branch name confirmed as main	Pass
4	Initial file commit	app.txt committed to main	Initial commit recorded on main	Pass
5	Feature branch creation	feature branch created from main	feature branch created successfully	Pass
6	Feature branch modification	app.txt updated on feature	Application Mode changed to Development	Pass

COMPLEX ENGINEERING PROBLEM - REPORT

Sr. No.	Test / Parameter	Expected Result	Obtained Result	Status
7	Main branch modification	app.txt updated on main	Application Mode changed to Production	Pass
8	Merge operation	Merge attempted from feature to main	Merge started, automatic merge not possible	Pass
9	Conflict detection	Git reports a content conflict	CONFLICT (content): Merge conflict in app.txt	Pass
10	Conflict marker inspection	Markers visible in app.txt	<<<<<<, =====, >>>>>> present in file	Pass
11	Conflict resolution	Final content decided and markers removed	Application Mode: Production retained, markers removed	Pass
12	Final merge commit	Merge commit created	Merge commit recorded successfully	Pass
13	Repository status verification	Clean working tree	"nothing to commit, working tree clean" shown	Pass
14	Commit history verification	Full branch/commit graph visible	git log --oneline --graph --all shows complete history	Pass
15	GitHub repository verification	Repository visible on GitHub	Repository and commit history visible after push	Pass

8. Challenges Faced and Solutions

Challenge	Solution
Understanding the purpose of branches	Studied that branches allow independent lines of development from a common commit history.
Creating a genuine conflict	Ensured that the same line of app.txt was edited differently in both branches before merging.
Understanding conflict markers	Reviewed Git documentation to learn the meaning of <<<<<<, =====, and >>>>>>.
Identifying current branch and incoming branch	Noted that HEAD refers to the branch being merged into (main) and the label after >>>>>> refers to the incoming branch (feature).
Deciding which content should remain after conflict	Compared both versions and selected the Production value, since main represents the stable branch.

COMPLEX ENGINEERING PROBLEM - REPORT

Challenge	Solution
Remembering to remove conflict markers	Carefully edited the file in Visual Studio Code and verified that no marker text remained.
Staging the resolved file	Used git add app.txt immediately after editing to mark the conflict as resolved.
Completing the merge	Used git commit without changing the default merge message to finalize the merge commit.
Verifying clean repository status	Ran git status after the commit to confirm the working tree was clean.
Understanding commit history	Used git log --oneline --graph --all to visualize how the branches diverged and were merged.

9. Conclusion

A Git repository was successfully created with a main branch and a feature branch, and both branches were used to modify the same line of app.txt in different ways. Merging the feature branch into main produced a genuine merge conflict, which was inspected using the conflict markers inserted by Git and resolved by deliberately choosing the Production value as the final content. The resolved file was staged and committed, and git status and git log were used to verify that the repository was clean and that the complete branch history was preserved. The final repository was pushed to GitHub and documented with the commit history. This exercise demonstrated how version control allows multiple developers to work independently while still providing a controlled and traceable mechanism to detect and resolve conflicting changes, which is an essential skill for collaborative software development.

10. SDG Mapping and Justification

SDG	Relevance to the Project	Justification
SDG 4 – Quality Education	Builds practical technical skills in version control and collaborative development	The project develops hands-on knowledge of Git branching, merge-conflict resolution, and open-source tools, which are core skills for a Computer Engineering student.
SDG 9 – Industry, Innovation and Infrastructure	Supports modern software engineering infrastructure and practices	The project uses an open-source distributed version control system that underpins collaborative software development in industry, supporting innovation in digital infrastructure.
SDG 17 – Partnerships for the Goals	Demonstrates collaborative development through shared, open-source tooling	Git and GitHub enable multiple developers to collaborate on a shared codebase, reflecting the

COMPLEX ENGINEERING PROBLEM - REPORT

SDG	Relevance to the Project	Justification
		spirit of partnership and collaboration that this SDG promotes.

11. References (in IEEE Style)

- [1] Git, "Git Documentation," git-scm.com. [Online]. Available: <https://git-scm.com/doc>
- [2] S. Chacon and B. Straub, Pro Git, 2nd ed. Berkeley, CA, USA: Apress, 2014. [Online]. Available: <https://git-scm.com/book/en/v2>
- [3] GitHub, Inc., "GitHub Docs," docs.github.com. [Online]. Available: <https://docs.github.com>